

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA



Tecnologias Para o Desenvolvimento Web

Raimundo Marcos Gonçalves Ludgério
Professor

E-mail: raimundo.marcos@ifpb.edu.br

Conteúdo programático

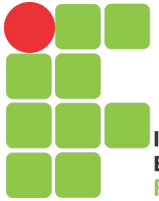
- Entender o que é programação assíncrona e por que ela é necessária
- Explicar o conceito de Promises e como usá-las.
- Utilizar os métodos .then(), .catch() e .finally().
- Escrever código assíncrono moderno com async/await.
- Tratar erros em operações assíncronas.

O QUE VIMOS NA AULA PASSADA?

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

- Revisão HTML/CSS;
- Document Object Model;
- Introdução a JavaScript;
- Princípios Básicos da Linguagem;
- Variáveis e constantes;
- Tipos Primitivos;
- Operadores;
- Condicionais;
- Funções.

REVISANDO...



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

COMO ADICIONAR JAVASCRIPT

- Utilizando a tag **<script>**
- Utilizando arquivo externo com extensão js
- No final do **<body> (melhora a performance)**

COMO ADICIONAR JAVASCRIPT

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

```
<body>
    <script>
        console.log("Olá, mmundo!")
    </script>
</body>
```

VARIÁVEIS E CONSTANTES

Existem três maneiras de declarar variáveis:

- **let:** variável de escopo
- **const:** constantes
- **var:** variáveis globais

Obs: O **;** no JavaScript é opcional.



Tipos Primitivos

- **String:** Sequência de texto
- **Number:** Valores numéricos
- **Boolean:** Verdadeiro ou falso
- **Array:** Estrutura que permite armazenar vários valores em um único referência
- **Object:** Tudo em JavaScript é um objeto e pode ser armazenados em uma variável

Operadores

Operador	Detalhes	Símbolo	Exemplo
Adição	Adicionar dois números. Concatenar duas Strings	+	5 + 6; “Olá” + “Mundo”;
Subtração, multiplicação e divisão	Exatamente a mesma coisa da matemática	-, *, /	9 - 3; 8 * 2; 15 / 5;
Atribuição	Guardar um valor e associar a uma referência	=	let nome = “Raimundo”;
Igualdade Restrita	Verifica se dois valores são iguais.	===	let num = 3; 3 === 4; (falso)
Negação	Utilizado para negar um valor booleano	!	!(3===4);
Diferente	Compara se dois valores não são iguais	!==	let nome = “a”; nome !== “b”;

Operadores

Operador	Detalhes	Símbolo	Exemplo
E lógico	Verdade quando as duas operações forem verdade	&&	true && true
OU lógico	Verdade quando pelo menos uma das operações forem verdades	||	false || true

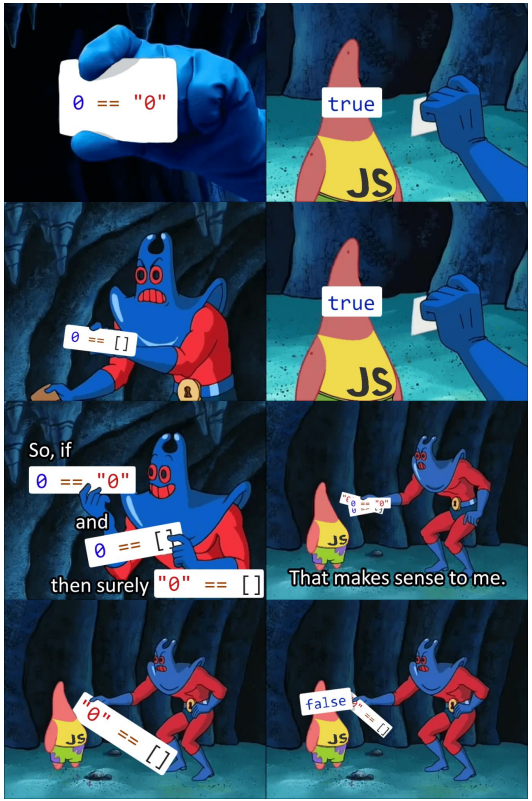
Operadores

JavaScript permite usar operadores lógicos com valores que não são booleanos.

O comportamento é baseado na ideia de valores true e false.

```
console.log(null || "valor padrão"); // "valor padrão"
console.log("texto" && 123);         // 123
console.log(0 && "oi");              // 0 (false)
console.log(!0);                     // true
```

Operadores


0 == "0"
true
JS
0 == []
true
JS
So, if
0 == "0"
and
0 == []
then surely "0" == []
That makes sense to me.
"0" == []
false

Condicionais

Condicionais são estruturas de código usadas para testar se uma expressão retorna verdadeira ou não.

Uma forma muito comum de condicionais é a instrução **if...else**.

Condicionais


```
<input type="text" name="nome" id="nome" placeholder="Informe o seu nome">
<script>
    let nome = document.getElementById("nome").value;
    if (nome === "Raimundo") window.alert("Olá, Raimundo. Você é o administrdor");
    else window.alert("Verifique os dados e tente novamente! ");
</script>
```

Switch/case

Para condições simples, o JavaScript contém a estrutura Switch/case.

```
let cor = "vermelho";
switch (cor) {
    case "azul":
        console.log("Cor do céu");
        break;
    case "vermelho":
        console.log("Cor do fogo");
        break;
    default:
        console.log("Outra cor");
}
```



LAÇOS

Laço for:

```
for (let i = 0; i < 5; i++) {
    console.log("Número: " + i);
}
```

LAÇOS

Laço while:

```
let contador = 0;
while (contador < 3) {
  console.log("Repetição " + contador);
  contador++;
}
```

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

LAÇOS

Laço do…while:

```
let i = 0;
do {
    console.log("Número:", i);
    i++;
} while (i < 5);
```

Funções

Functions são uma forma de empacotar a funcionalidade que são reutilizáveis.

É possível definir um bloco de código como uma função que é executada quando é chamado pelo nome da função.

FUNÇÕES

- **getElementById():** Retorna o elemento HTML com o ID especificado
- **getElementsByClassName():** Retorna uma coleção de elementos que possuem a classe especificada
- **getElementsByTagName():** Retorna uma coleção de todos os elementos com a tag HTML especificada (por exemplo, div, p, a)



FUNÇÕES

- **querySelector():** Retorna o elemento no documento que corresponde ao seletor fornecido.
- **querySelectorAll():** Retorna uma NodeList (uma coleção de nós) de todos os elementos no documento que correspondem ao seletor fornecido.

FUNÇÕES


```
// Seleciona o primeiro parágrafo com a classe .paragrafo
let primeiroParagrafo = document.querySelector('.paragrafo');
console.log(primeiroParagrafo.textContent);


// Seleciona o elemento com o id 'titulo'
let titulo = document.querySelector('#titulo');
console.log(titulo.textContent);

// Seleciona o primeiro <span>
let span = document.querySelector('span');
console.log(span.textContent);
```

FUNÇÕES


```
let titulo = document.getElementById('titulo');
console.log(titulo.textContent);

let paragrafos = document.getElementsByClassName('paragrafo');
console.log(paragrafos[0].textContent); // Parágrafo 1
console.log(paragrafos[1].textContent); // Parágrafo 2

let spans = document.getElementsByTagName('span');
console.log(spans[0].textContent); // Texto dentro do span
```

ARGUMENTOS

Os argumentos ficam dentro dos parênteses, separados por vírgulas se houver mais de um argumento.

```
function multiplicacao(num1, num2) {
    let result = num1 * num2;
    return result;
}
multiplicacao(8,2)
```

ARGUMENTOS OPCIONAIS


```
function multiplicacaoOpc(num1 = 0, num2 = 1) {
    let result = num1 * num2;
    return result;
}

console.log(multiplicacaoOpc(8))
```

VARIÁVEIS FUNÇÕES

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

O JS permite armazenar funções em variáveis

```
const minhaFun = function (numero) {
    return numero * 3
}

console.log(minhaFun(8))
```

Funções Anônimas

Funções anônimas são funções que não possuem um nome **identificador**.

Elas são declaradas sem um nome e geralmente são usadas em contextos onde a função é usada como valor ou passada como argumento para outras funções.

Funções Anônimas



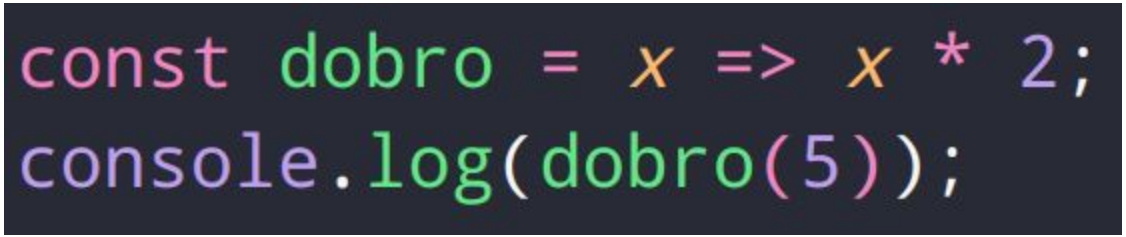
- Não possuem nome: São definidas sem um identificador
- São expressões: Podem ser atribuídas a variáveis ou passadas como argumentos
- Úteis para callbacks: Muito usadas em eventos e operações assíncronas

Funções Anônimas


```
const funAnonima = () => {
    console.log("Essa é uma arrow function")
}

array.map(function(item) {
    return item * 2;
})
```

Funções Anônimas





Eventos

São estruturas de código que detectam atividades no navegador e executam o código em resposta.

exemplo é lidar com o evento de **click**, que é acionado pelo navegador quando o usuário clica em algo com o mouse.

Eventos


```
<!DOCTYPE html>
<html lang="pt-br">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Introdução a Javascript</title>
</head>

<body>
    <script>
        document.querySelector("html").addEventListener("click", function () {
            alert("Ai! Pare de me cutucar!");
        });
    </script>
</body>
</html>
```

Eventos em botões e formulários



O JavaScript permite interagir dinamicamente com elementos da DOM.

Portanto, é possível adicionar eventos a botões, textos, mensagens e etc.

Eventos


```
<body>
    <button id="meuBotao">Clique aqui</button>
    <input type="text" id="meuInput" placeholder="Digite algo">
    <div id="info"></div>
    <script>
        document.getElementById("meuBotao").addEventListener("click", () => {
            let valor = document.getElementById("meuInput").value;
            document.getElementById("info").innerHTML = valor;
        });
    </script>
</body>
```

Orientação a Objetos

Um objeto é uma coleção de dados e/ou funcionalidades relacionadas (que geralmente consistem em diversas variáveis e funções – que são chamadas de propriedades e métodos quando estão dentro de objetos).

Orientação a Objetos

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

O JavaScript também implementa o paradigma orientado a objetos.

Portanto, é possível construir sistemas trabalhando com os pilares da orientação a objetos: herança, polimorfismo, encapsulamento e abstração.

Orientação a Objetos

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Pode-se utilizar os objetos de maneira tradicional:

```
export default class Pessoa {
    constructor(nome, idade) {
        this.nome = nome;
        this.idade = idade;
    }

    apresentar() {
        console.log(`Olá, meu nome é ${this.nome} e tenho ${this.idade} anos.`);
    }
}
```

Orientação a Objetos

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Pode-se utilizar os objetos de maneira tradicional:

```
import Pessoa from "./Pessoa.js";

var p1 = new Pessoa();
p1.idade = 37;
p1.nome = "Maria";
p1.apresentar();
```

Orientação a Objetos

Podemos trabalhar com JSON (JavaScript Object Notation)



Orientação a Objetos


```
var pessoa = {
    nome: ["Bob", "Smith"],
    idade: 32,
    sexo: "masculino",
    interesses: ["música", "esquiar"],
    bio: function () {
        alert(
            this.nome[0] +
            " " +
            this.nome[1] +
            " tem " +
            this.idade +
            " anos de idade. Ele gosta de " +
            this.interesses[0] +
            " e " +
            this.interesses[1] +
            ".",
        );
    },
    saudacao: function () {
        alert("Oi! Eu sou " + this.nome[0] + ".");
    },
};
```

Orientação a Objetos


```
pessoa.nome || pessoa["nome"];
pessoa.nome[0];
pessoa.idade;
pessoa.interesses[1];
pessoa.bio();
pessoa.saudacao();
```

Métodos de arrays

ARRAY HELPERS (MÉTODOS DE ARRAY)



Eles permitem aplicar uma função a cada elemento sem precisar escrever o laço for manualmente.

Eles retornam novos valores, sem modificar o array original.

ARRAY HELPERS (MÉTODOS DE ARRAY)


```
const pessoas = [
    { nome: "Alice", idade: 28, cidade: "São Paulo" },
    { nome: "Bruno", idade: 34, cidade: "Rio de Janeiro" },
    { nome: "Carla", idade: 22, cidade: "Belo Horizonte" },
    { nome: "Diego", idade: 40, cidade: "Curitiba" },
    { nome: "Eduarda", idade: 19, cidade: "Porto Alegre" },
    { nome: "Fábio", idade: 31, cidade: "Recife" },
    { nome: "Gabriela", idade: 27, cidade: "Fortaleza" },
    { nome: "Henrique", idade: 45, cidade: "Manaus" },
    { nome: "Isabela", idade: 23, cidade: "Salvador" },
    { nome: "João", idade: 37, cidade: "Brasília" }
];
```

ARRAY HELPERS (MÉTODOS DE ARRAY)

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
PARAÍBA

Vamos criar um programa que imprima cada pessoa:

```
for(p of pessoas){
    console.log(p["nome"]);
    console.log(p.idade)
}
```

ARRAY HELPERS (MÉTODOS DE ARRAY)

Agora vamos selecionar apenas as pessoas que tem mais de 20 anos.

```
let pessoas_mais_20 = []
for(p of pessoas){
    if(p.idade >= 20) pessoas_mais_20.push(p)
}
for(p of pessoas_mais_20){
    console.log(p)
}
```

ARRAY HELPERS (MÉTODOS DE ARRAY)

- Para esses cenários, o JS disponibiliza alguns métodos para trabalhar com arrays
- array.**forEach(function())** -> varre todos os elementos do array
- array.**find(function())** -> Retorna o **primeiro** elemento que satisfaz a condição de teste.

ARRAY HELPERS (MÉTODOS DE ARRAY)

- array.**some(function())** -> Verifica se pelo menos um elemento satisfaz a condição
- array.**every(function())** -> Verifica se todos os elementos satisfazem a condição.

ARRAY HELPERS (MÉTODOS DE ARRAY)

- array.**map(function())** -> percorre cada elemento do array, aplica uma função transformadora a ele, e retorna um novo array com os resultados
- array.**filter(function())** -> percorre cada elemento do array e retorna um novo array contendo somente os elementos para os quais o teste retornou true.



forEach()

- Não retorna nada (undefined)
- Não modifica o array original (a menos que você modifique dentro da função)
- Não pode ser interrompido com break ou return

forEach()


```
pessoas.forEach((pessoa, indice) => {
    console.log(`A pessoa ${pessoa.nome} está na posição ${indice}`)
})
```

```
pessoas.forEach((pessoa, indice) => console.log(`${pessoa.nome} - ${indice}`))
```

Varre todos os elementos.



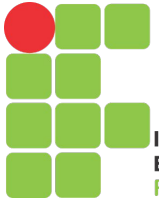
FIND()

- Retorna o elemento encontrado
- Retorna undefined se nenhum elemento satisfazer a condição
- Para no primeiro elemento que satisfaz a condição

FIND()


```
let maioridade = pessoas.find(pessoa => pessoa.idade >= 18)
console.log(maioridade)
```

```
let maioridade = pessoas.find(function (pessoa) {
    return pessoa.idade >= 18;
})
console.log(maioridade)
```



some()

- Retorna boolean (true/false)
- Para na primeira ocorrência que satisfaz a condição
- Útil para validações

SOME()


```
const comecaComA = pessoas.some(pessoa => pessoa.nome.charAt(0).toUpperCase() === "A");
console.log(comecaComA) // true
```

```
const comecaComA = pessoas.some(function(pessoa) {
    return pessoa.nome.charAt(0).toUpperCase() === "A";
});
console.log(comecaComA) // true
```



EVERY()

- Retorna boolean (true/false)
- Para no primeiro elemento que não satisfaz a condição
- Array vazio retorna true (condição vacuamente verdadeira)

EVERY()


```
const comecaComA = pessoas.every(pessoa => pessoa.nome.charAt(0).toUpperCase() === "A");
console.log(comecaComA) // false
```

```
const comecaComA = pessoas.every(function(pessoa) {
    return pessoa.nome.charAt(0).toUpperCase() === "A";
});
console.log(comecaComA) // false
```



MAP()

- Percorre cada elemento do array
- Aplica uma função transformadora a ele
- Retorna um novo array com os resultados
- O array original **não é alterado** (a menos que você faça modificações explícitas nos objetos internos)
- Tamanho do novo array = tamanho do array original.

MAP()


```
const nome_pessoas = pessoas.map(function(pessoa){
    return pessoa.nome
});
console.log(nome_pessoas);
```

```
const nome_pessoas = pessoas.map(pessoa => pessoa.nome);
console.log(nome_pessoas);
```



FILTER()

- Percorre cada elemento do array
- Aplica uma função de teste (predicate) que retorna true ou false para cada elemento
- Retorna um novo array contendo somente os elementos para os quais o teste retornou true

FILTER()


```
const pessoas_de_jampa = pessoas.filter(p => p.cidade == "João Pessoa");
console.log(pessoas_de_jampa)
```

```
const pessoas_de_jampa = pessoas.filter(function (p) {
    return p.cidade == "João Pessoa"
});
console.log(pessoas_de_jampa)
```

ATIVIDADE

Jogo de Par ou Ímpar contra a Máquina

Criar uma página que permita jogar Par ou Ímpar contra a máquina.

- O usuário escolhe Par ou Ímpar
- O usuário digita um número
- A máquina escolhe um número aleatório entre 0 e 10
- Soma-se os dois números.
- Se o resultado for par e o usuário tiver escolhido Par, ele vence (e vice-versa).

Jogo de Par ou Ímpar contra a Máquina



- A atividade deve ser feita individualmente.
- Não é permitido consultar internet, IA, colegas ou materiais externos.
- É permitido apenas utilizar o editor de código e o navegador para testar o programa.
- A entrega deve conter um único arquivo HTML com o código JavaScript embutido.
- Ao final do tempo, o aluno deve entregar o arquivo conforme instrução do professor.

PROGRAMAÇÃO ASSÍNCRONA

PROGRAMAÇÃO ASSÍNCRONA



Um **paradigma de programação** que permite que um programa inicie tarefas potencialmente demoradas e continue executando outras operações sem esperar pela conclusão da primeira tarefa.

PROGRAMAÇÃO SÍNCRONA

Em contraste com a **programação síncrona**, onde uma tarefa **deve ser concluída** para que a próxima comece, a programação assíncrona executa tarefas em paralelo, evitando bloqueios e permitindo uma melhor utilização dos recursos do sistema.

Como funciona a programação assíncrona

Início de tarefa: O programa inicia uma operação de longa duração (como uma requisição de rede) sem bloquear o fluxo principal.

Continuação de tarefas: A execução continua com outras tarefas, como responder a interações do usuário ou executar outros processos.

Como funciona a programação assíncrona

Conclusão e retorno: Quando a tarefa demorada é concluída, o programa a gerencia e processa seu resultado, muitas vezes usando mecanismos como async/await para um código que parece síncrono.

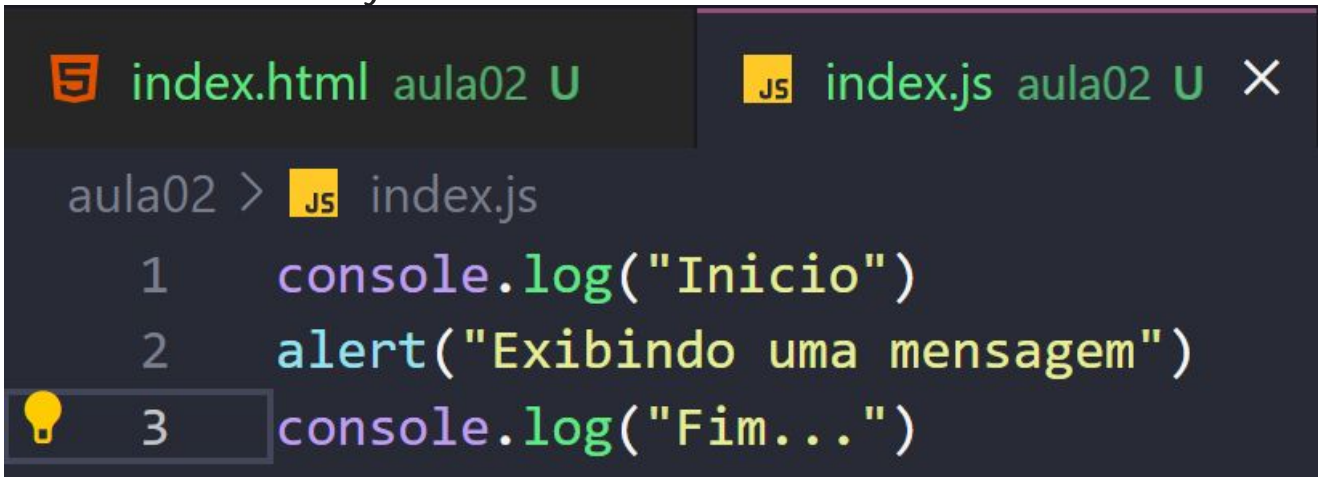
O PROBLEMA DA EXECUÇÃO SÍNCRONA

O JavaScript é **single-threaded**, ou seja, executa uma tarefa por vez.

Isso significa que, se uma operação demorar (como uma requisição de rede), ela bloqueia o restante do código.

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

O PROBLEMA DA EXECUÇÃO SÍNCRONA



Enquanto o **alert()** está ativo, o programa para completamente.

Solução: Programação Assíncrona

Permite executar tarefas demoradas sem bloquear o fluxo principal.

Principais mecanismos:

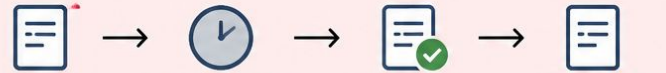
- Callbacks
- Promises
- Async/Await

Programação Assíncrona em JavaScript

Permite iniciar uma tarefa que pode levar algum tempo para terminar sem bloquear a execução do restante do código. Enquanto a tarefa é executada, o programa continua fazendo outras atividades e, quando ela termina, o resultado é processado.

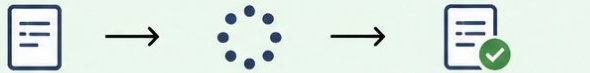
Execução Síncrona (bloqueante)

O programa fica esperando a tarefa terminar para continuar com as próximas instruções.



Execução Assíncrona (não bloqueante)

O programa inicia a tarefa e continua executando outras instruções. Quando a tarefa termina, o resultado é processado.



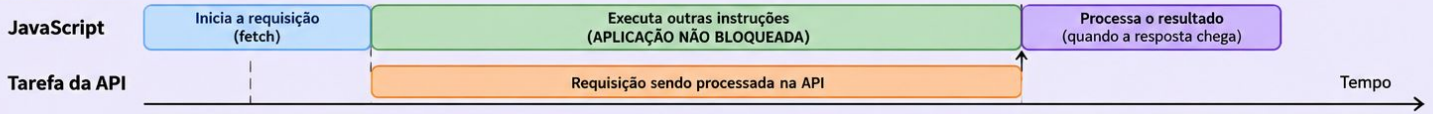
Exemplo em JavaScript (com async/await)

```
1  async function carregarUsuarios() {
2      console.log('Iniciando requisição para a API...');
3
4      try {
5          const resposta = await fetch('https://jsonplaceholder.typicode.com/users');
6          const usuarios = await resposta.json();
7          console.log('Resposta recebida!');
8          console.log('Primeiro usuário:', usuarios[0]);
9      } catch (erro) {
19         console.error('Erro ao buscar usuários:', erro);
11     }
11 }
12 carregarUsuarios();
```

Passo a passo do exemplo

1. **Inicia a requisição**
   A função *fetch()* envia uma requisição para a API.
2. **Não bloqueia a aplicação**
   Enquanto espera a resposta, o JavaScript pode executar outras tarefas (ou o restante da aplicação).
3. **Recebe a resposta**
   Quando a resposta chega, o *await* continua e o resultado é processado.
4. **Usa o resultado**
   O resultado da API é convertido para JSON e pode ser usado na aplicação.

Linha do tempo da execução assíncrona



CALLBACKS

Callbacks

Funções **passadas como argumento** para outras funções, executadas quando uma operação é concluída.

O programador define uma função e a passa como um argumento para outra função.

Callbacks

```
function funcaoBase(callback){
    setTimeout(() => {
        callback();
    }, 2000);
};

funcaoBase(() => {
    console.log("Mensagem que aparece depois de 2s");
});
```

Callbacks - com argumentos

```
function funcaoBase(callback){
    setTimeout(() => {
        callback("Argumento passado...");
    }, 2000);
};

funcaoBase((argumento) => {
    console.log(argumento);
});
```

Callback Hell

```
fazerLogin((usuario) => {
    buscarDadosUsuario(usuario.id, (dados) => {
        carregarPosts(dados.categoria, (posts) => {
            renderizarPosts(posts, () => {
                console.log("Finalmente tudo carregou!");
            });
        });
    });
});
```

Callback Hell

```
console.log("Início do processo...");
setTimeout(() => {
    console.log("Etapa 1 concluída");
    setTimeout(() => {
        console.log("Etapa 2 concluída");
        setTimeout(() => {
            console.log("Etapa 3 concluída");
            setTimeout(() => {
                console.log("Etapa 4 concluída");
                console.log("Processo finalizado.");
            }, 1000);
        }, 1000);
    }, 1000);
}, 1000);
```

Callback Hell

```
console.log("Início do processo...");
setTimeout(() => {
    console.log("Etapa 1 concluída");
    setTimeout(() => {
        console.log("Etapa 2 concluída");
        setTimeout(() => {
            console.log("Etapa 3 concluída");
            setTimeout(() => {
                console.log("Etapa 4 concluída");
                console.log("Processo finalizado.");
            }, 1000);
        }, 1000);
    }, 1000);
}, 1000);
```

Como evitar callback hell?

- Promises
- Async/Await

PROMISES

Promises

Uma **Promise** é um objeto que representa uma operação assíncrona que pode:

- Pendências (pending): ainda em execução
- Cumprida (fulfilled): terminou com sucesso
- Rejeitada (rejected): falho

Promises

Permite que o código lide com o resultado de uma tarefa que ainda não terminou quando a função foi chamada.

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Promises

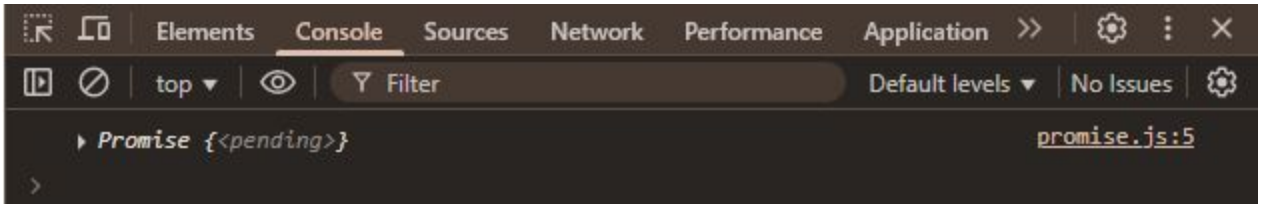
```
// carregando usuários de uma API externa (exemplo)
const usuarios = fetch(
  "https://raw.githubusercontent.com/raimundoludgerio/material-didatico/
  refs/heads/main/web/javascript/aula02/usuarios.json",
);
// fetch retorna uma Promise
console.log(usuarios);
```

Promises

Em uma **requisição HTTP**, enviamos uma solicitação para o servidor e ele nos envia uma resposta de volta.

Neste caso, vamos enviar uma **solicitação** para obter um arquivo **JSON** do servidor.

Promises


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
Promise {<pending>}
promise.js:5

Promises

```
// carregando usuários de uma API externa (exemplo)
const usuariosPromise = fetch(
  "https://raw.githubusercontent.com/raimundoludgerio/material-didatico/
  refs/heads/main/web/javascript/aula02/usuarios.json",
);
// fetch retorna uma Promise
usuariosPromise.then(usuarios => {
    console.log(usuarios);
});
```

Promises


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
promise.js:7
Response {type: 'cors', url: 'https://raw.githubusercontent.com/raimundoludgerio…fs/heads/main/web/javascript/aula02/usuarios.json', redirected: false, status: 200, ok: true, …}

INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA PARAÍBA

Promises

O retorno da requisição retorna uma **Response** que é uma interface da **Fetch API** que representa a resposta de uma requisição de rede.

Para interpretar a resposta iremos utilizar a função **json()** que converte Response para Objeto Javascript (json).

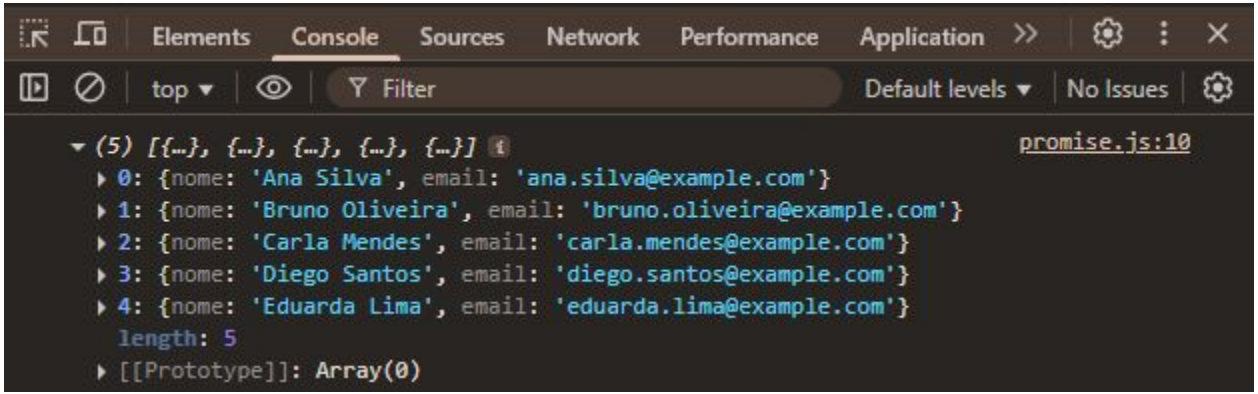
Promises

Acontece que **json()** também é assíncrono. Portanto, este é um caso em que temos que chamar duas funções assíncronas sucessivas.

Promises

```
// fetch retorna uma Promise
usuariosPromise.then(usuarios => {
    const usuariosJsonPromise = usuarios.json();
    // json também retorna uma Promise
    usuariosJsonPromise.then(usuarios => {
        console.log(usuarios);
    });
});
```

Promises


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
(5) [{…}, {…}, {…}, {…}, {…}]
promise.js:10
0: {nome: 'Ana Silva', email: 'ana.silva@example.com'}
1: {nome: 'Bruno Oliveira', email: 'bruno.oliveira@example.com'}
2: {nome: 'Carla Mendes', email: 'carla.mendes@example.com'}
3: {nome: 'Diego Santos', email: 'diego.santos@example.com'}
4: {nome: 'Eduarda Lima', email: 'eduarda.lima@example.com'}
length: 5
[[Prototype]]: Array(0)

Promises

Como as Promises podem ser aninhadas, é possível encadear várias Promises, como fizemos com as Callbacks.

Como a função **then()** também retorna uma Promise, pode-se simplificar o código aninhando os retornos.

Promises

```
// Aninhando várias promises
usuariosPromise.then(response => response.json()
    .then(usuarios =>console.log(usuarios))
);
```

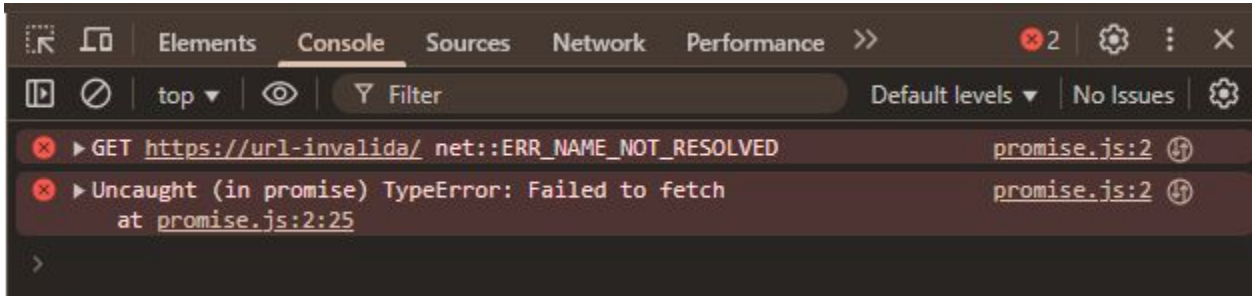
FUNÇÃO CATCH

A **API fetch()** pode lançar um erro por vários motivos (por exemplo, porque não havia conectividade de rede ou a URL estava mal formada de alguma forma) e estamos lançando um erro nós mesmos se o servidor retornar um erro.

FUNÇÃO CATCH

```
const usuariosPromise = fetch(
    "https://url-invalida",
);
// Aninhando várias promises
usuariosPromise.then(response => response.json(
    .then(usuarios =>console.log(usuarios))
);
```

FUNÇÃO CATCH



FUNÇÃO CATCH

Para suportar o **tratamento de erros**, os objetos Promise fornecem um método **catch()**.

Que é muito parecido com then(): você chama e passa uma função manipuladora.

FUNÇÃO CATCH

No entanto, enquanto o manipulador passado para **then()** é chamado quando a operação assíncrona é bem-sucedida, a função manipuladora passado para **catch()** é chamado quando a operação assíncrona falha.

FUNÇÃO CATCH

```
usuariosPromise.then(response => response.json()
    .then(usuarios =>console.log(usuarios))
).catch(error => console.log(error));
```

FUNÇÃO FINALLY

Serve para executar um **código de limpeza** ou **finalização sempre**, independentemente de a promessa ter sido resolvida com sucesso ou rejeitada por um erro.

FUNÇÃO FINALLY

É usada para realizar tarefas que precisam **acontecer sempre**, como parar um indicador de carregamento ou fechar uma conexão, antes de passar o resultado (ou erro) para o próximo manipulador, como **.then()** ou **.catch()**

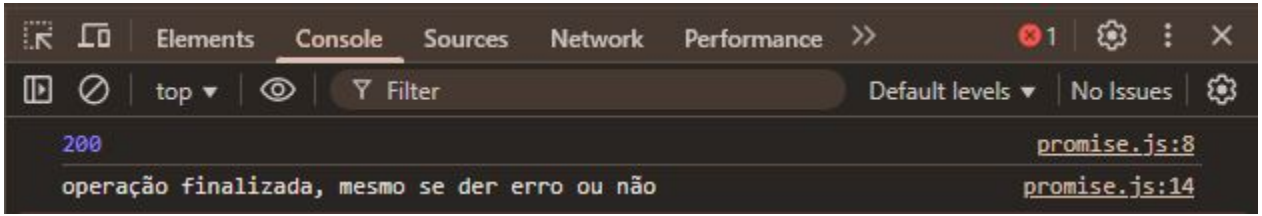
FUNÇÃO FINALLY

A função de callback fornecida ao **finally()** é executada em ambos os cenários: quando a promessa é **completada** com sucesso (resolvida) e quando ela **falha** (rejeitada).

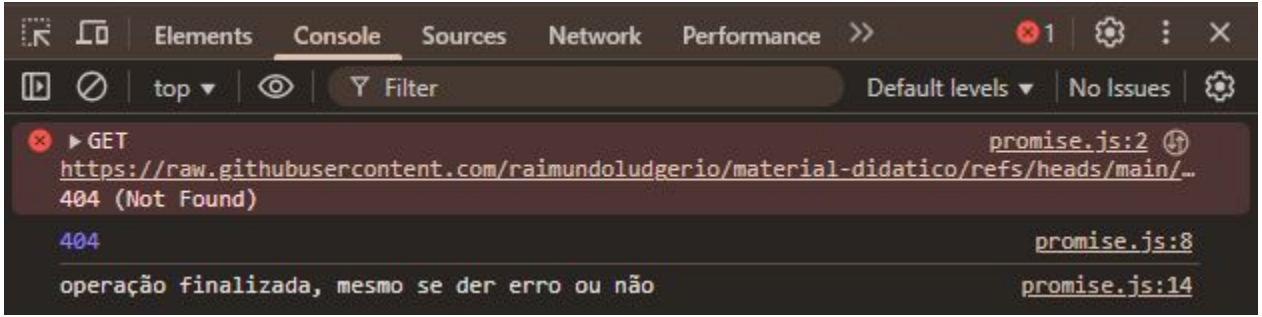
FUNÇÃO FINALLY

```
usuariosPromise.then(response => {
    // Caso de sucesso (resolve)
    console.log(response.status);
}).catch(error => {
    // Caso de sucesso (reject)
    console.log(error);
}).finally(() => {
    // Executa sempre
    console.log("operação finalizada, mesmo se der erro ou não");
});
```

FUNÇÃO FINALLY



FUNÇÃO FINALLY



CRIANDO PROMISE

Para criar uma **Promise** em JavaScript, use o construtor **new Promise()** passando uma função que recebe **resolve** e **reject** como argumentos.

CRIANDO PROMISE

Dentro dessa função executora, colocamos a lógica assíncrona, e quando o resultado desejado for obtido, chame **resolve(valor)** para o sucesso, ou **reject(erro)** para falha.

O valor passado para **resolve** é recebido no método **.then()** e o valor passado para **reject** é tratado pelo método **.catch()**.

CRIANDO PROMISE

```
const minhaPromise = new Promise((resolve, reject) => {
    // fazendo qualquer coisa assíncrona
});
```

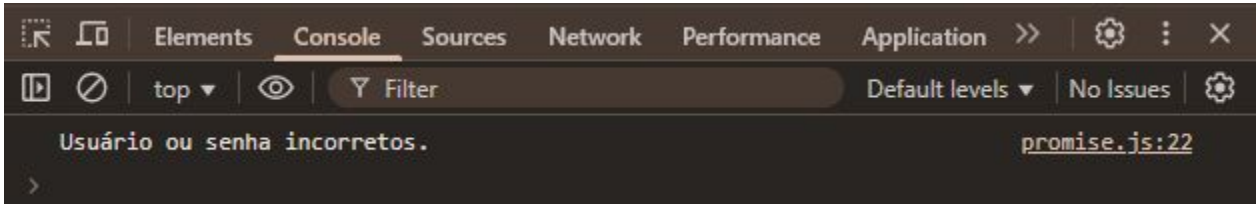
CRIANDO PROMISE


```
const minhaPromise = (usuario, senha) => new Promise((resolve, reject) => {
    setTimeout(() => {
      if (usuario === "admin" && senha === "1234") {
        resolve("Login bem-sucedido!");
      } else {
        reject("Usuário ou senha incorretos.");
      }
    }, 2000); // 2 segundos de "atraso"
});
```

CRIANDO PROMISE

```
minhaPromise("marcos", "123").then(sucesso => {
    console.log(sucesso);
}).catch(falha => {
    console.log(falha);
});
```

CRIANDO PROMISE



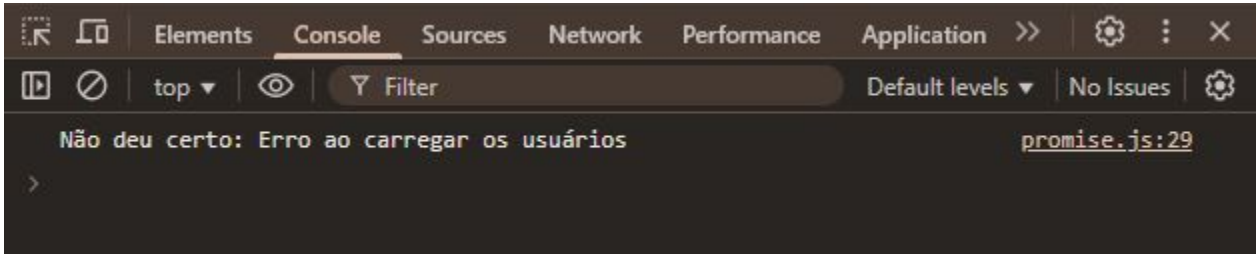
CRIANDO PROMISE

```
const promiseCarregandoUsuarios = new Promise((resolve, reject) => {
    setTimeout(() => {
        const sucesso = Math.random() * 10 >= 5;
        if(sucesso) resolve("Usuários carregdos");
        else reject("Erro ao carregar os usuários");
    }, 2000);
});
```

CRIANDO PROMISE

```
promiseCarregandoUsuarios.then(resultado => {
    console.log("Deu certo: " + resultado);
}).catch(falha => {
    console.log("Não deu certo: " + falha);
});
```

CRIANDO PROMISE



TRABALHANDO COM MÚLTIPLAS PROMISES

Quando trabalha-se com múltiplas Promises simultâneas, faz-se necessário a utilização de funções específicas para trabalhar com elas: **race()** e **all()**.

Promise.all()

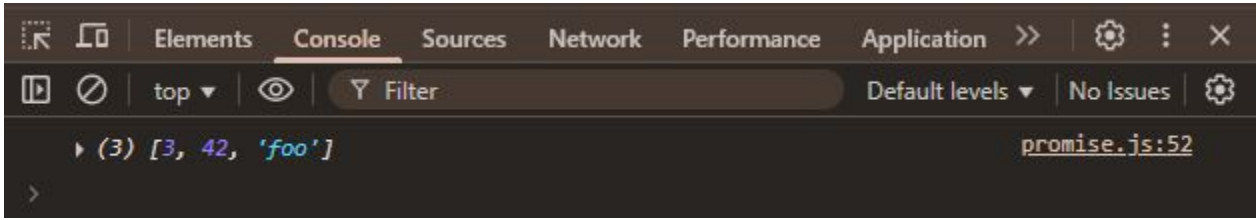
Espera todas as Promises terminarem:

```
const promise1 = Promise.resolve(3);
const promise2 = 42; // Valor não-promise é convertido
const promise3 = new Promise((resolve) => {
    setTimeout(resolve, 1000, "foo");
});

Promise.all([promise1, promise2, promise3])
    .then((valores) => {
        console.log(valores); // [3, 42, "foo"] após 1 segundo
    })
    .catch(erro => {
        console.error("Uma das promises falhou:", erro);
    });
```

Promise.all()

Espera todas as Promises terminarem:



Promise.race()

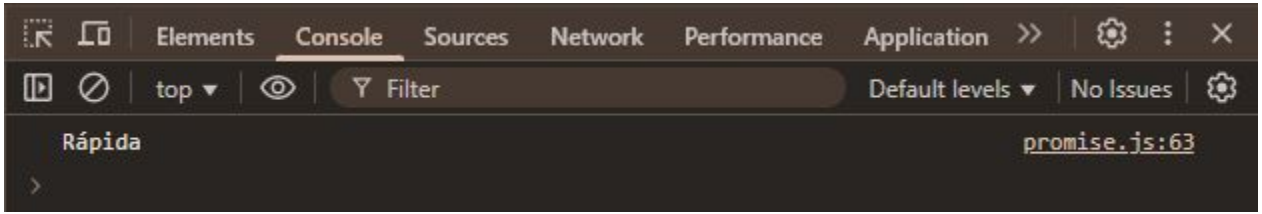
Primeira promise a ser resolvida/rejeitada

```
const promiseRapida = new Promise(resolve => setTimeout(resolve, 1000, "Rápida"));
const promiseLenta = new Promise(resolve => setTimeout(resolve, 3000, "Lenta"));

Promise.race([promiseRapida, promiseLenta])
    .then(vencedora => {
        console.log(vencedora); // "Rápida" (após 1s)
    });
```

Promise.race()

Primeira promise a ser resolvida/rejeitada



async/await

ASYNC/AWAIT

Recurso sintático que permite escrever código **assíncrono** de forma que se pareça e se comporte como código **síncrono**, mas sem bloquear o fluxo principal do programa.

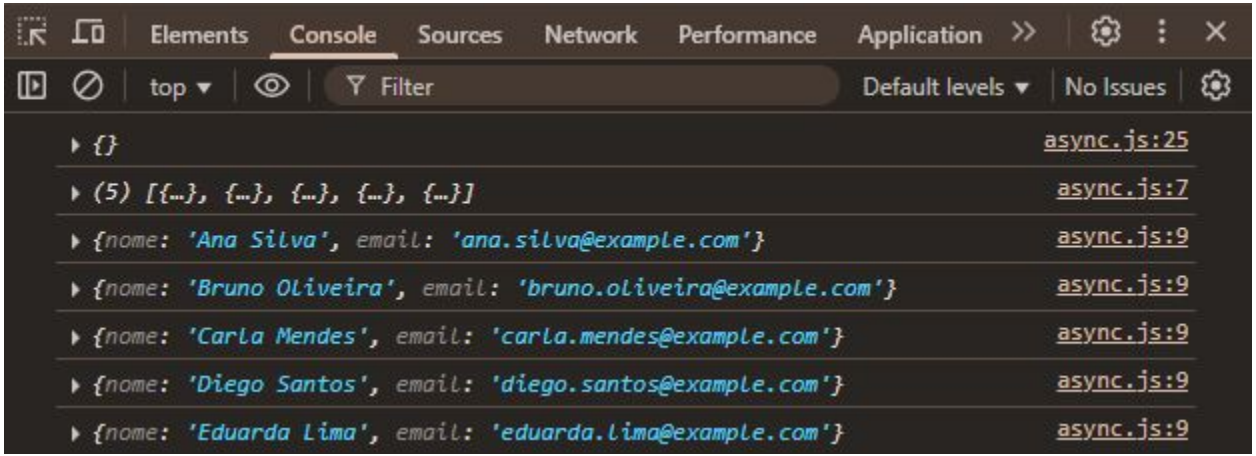
ASYNC/AWAIT

A palavra-chave **async** declara uma função que sempre retornará uma **Promise**, enquanto **await** é usada dentro dessas funções para **pausar** a execução até que uma **Promise** seja resolvida, retornando seu valor ou lançando um erro.

ASYNC/AWAIT

```
function minhaFuncaoAssincrona(user) {
    const buscaApi = fetch("https://raw.githubusercontent.com/raimundoludgerio/material-didatico/refs/heads/main/web/
    javascript/aula02/usuarios.json");
    let userFound = {}
    buscaApi.then(res => {
        const users = res.json();
        users.then(usuarios => {
            console.log(usuarios)
            usuarios.forEach(u => {
                console.log(u)
                if(u.email == user.email) {
                    userFound = u
                };
            });
        });
    });
    return userFound;
}
```

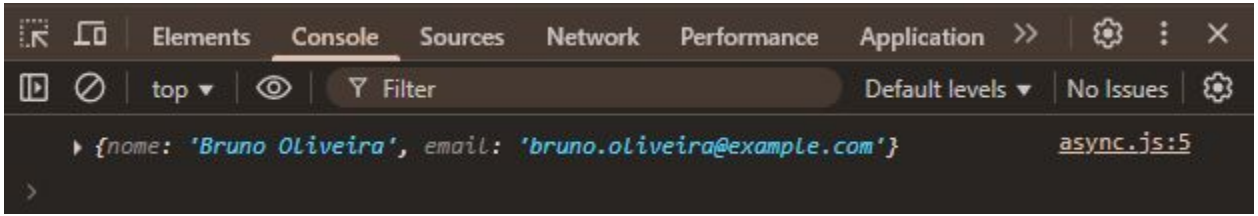
ASYNC/AWAIT


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
{}
async.js:25
(5) [{…}, {…}, {…}, {…}, {…}]
async.js:7
{nome: 'Ana Silva', email: 'ana.silva@example.com'}
async.js:9
{nome: 'Bruno Oliveira', email: 'bruno.oliveira@example.com'}
async.js:9
{nome: 'Carla Mendes', email: 'carla.mendes@example.com'}
async.js:9
{nome: 'Diego Santos', email: 'diego.santos@example.com'}
async.js:9
{nome: 'Eduarda Lima', email: 'eduarda.lima@example.com'}
async.js:9

ASYNC/AWAIT


```
async function minhaFuncaoAssincrona(user) {
    const buscaApi = await fetch("https://raw.githubusercontent.com/raimundoludgerio/material-didatico/refs/heads/
    main/web/javascript/aula02/usuarios.json");
    const res = await buscaApi.json()
    res.forEach(u => {
        if(u.email === user.email) console.log(u);
    });
}
```

ASYNC/AWAIT


Elements
Console
Sources
Network
Performance
Application
top
Filter
Default levels
No Issues
{nome: 'Bruno Oliveira', email: 'bruno.oliveira@example.com'}
async.js:5

async/await - Tratamento de Erros

O tratamento de erros com **async/await** é feito usando um bloco **try...catch**, que permite capturar e gerenciar exceções de forma limpa e familiar, simplificando o tratamento de erros em operações assíncronas.

ASYNC/AWAIT - TRATAMENTO DE ERROS


```
async function essaFuncaoTemErro() {
    try {
        const buscaApi = await fetch("url_invalida");
        if (!buscaApi.ok) throw new Error("Requisição Falhou");
        const res = await buscaApi.json()
        return res;
    } catch (erro) {
        console.error("Erro capturado: " + erro)
    }
}
```

async/await - Tratamento de Erros

O tratamento de erros com **async/await** é feito usando um bloco **try...catch**, que permite capturar e gerenciar exceções de forma limpa e familiar, simplificando o tratamento de erros em operações assíncronas.

ATIVIDADE

Consumindo e Exibindo Dados



Criar uma **página web** estilizada com HTML + CSS (ou Tailwind/Bootstrap) que consuma dados da URL fornecida e apresente as informações dinamicamente no navegador utilizando **JavaScript assíncrono** e **manipulação do DOM**.

Consumindo e Exibindo Dados



- Fazer requisições HTTP usando fetch() e Promises/async-await:
  - https://jsonplaceholder.typicode.com/users
- Manipular e exibir dados dinâmicos no DOM.
- Aplicar boas práticas de estrutura e estilo (HTML semântico + CSS framework).
- Lidar com erros de rede e respostas inválidas.

Funcionalidades mínimas

- Página HTML com layout organizado:
  - Cabeçalho (título e breve descrição)
  - Área principal para exibir os dados da API
  - Rodapé simples
- Botão de interação com o usuário:
  - Carregar dados, buscar…
- Requisição à URL
- Exibição dos dados

OBRIGADO!

Repositório com os códigos: https://github.com/raimundoludgerio/estrutura-de-dados/tree/main/web

MDN contributors. **Introdução ao JavaScript Async**. 2025. Disponível em: https://developer.mozilla.org/pt-BR/docs/Learn_web_development/Extensions/Async_JS/Introducing#:~:text=A%20programa%C3%A7%C3%A3o%20ass%C3%ADncrona%20%C3%A9%20uma,que%20essa%20tarefa%20seja%20conclu%C3%ADda.

Contato:

raimundo.marcos@ifpb.edu.br

As Dúvidas podem ser enviadas pelo e-mail, conversar pessoalmente após a aula ou em algum horário a combinar.